

TLUX Backend

A Django REST Framework backend service built with modern best practices for scalability, security, and maintainability.

Overview

This project is a RESTful API backend built with Django 5.2 and Django REST Framework. It features JWT authentication, comprehensive API documentation, and containerized deployment with Docker for both development and production environments.

Features

- **RESTful API:** Built with Django REST Framework for robust API development
- **JWT Authentication:** Secure token-based authentication using `django-rest-framework-simplejwt`
- **API Documentation:** Auto-generated OpenAPI/Swagger documentation via `drf-spectacular`
- **CORS Support:** Configurable Cross-Origin Resource Sharing for frontend integration
- **Security Enhancements:** Admin honeypot protection and security headers
- **Comprehensive Logging:** Configurable logging system supporting console and file output with log rotation
- **Testing Framework:** pytest with Django integration, coverage reporting, and reusable fixtures
- **Database:** PostgreSQL with Django ORM
- **Task Queue:** Celery with RabbitMQ for asynchronous and periodic tasks
- **Containerized:** Docker and Docker Compose for easy deployment
- **Production Ready:** uWSGI application server with Nginx reverse proxy
- **Multi-Stage Builds:** Optimized Docker images for production

Technology Stack

- **Framework:** Django 5.2.7
- **API:** Django REST Framework 3.16.1
- **Authentication:** `django-rest-framework-simplejwt` 5.5.1
- **Documentation:** `drf-spectacular` 0.29.0
- **Database:** PostgreSQL 15
- **Application Server:** uWSGI 2.0.31
- **Web Server:** Nginx 1.25
- **Task Queue:** Celery 5.4.0 with RabbitMQ 3-management
- **Python:** 3.11 (Development), 3.13 (Production)

Prerequisites

- Docker 20.10+ and Docker Compose 2.0+
- Python 3.11+ (for local development without Docker)
- PostgreSQL 15+ (for local development without Docker)

Project Structure

```
tlux-backend/  
├── config/                                # Django project configuration  
│   ├── settings/                         # Environment-based settings  
│   │   ├── base.py                      # Base settings  
│   │   └── sub_settings/                # Modular configuration modules  
│   │       └── logging_conf.py          # Logging configuration  
│   ├── urls.py                          # Main URL routing  
│   └── wsgi.py                          # WSGI application entry point  
├── core/                                # Core utilities  
│   ├── logging.py                       # Logging utilities and helpers  
│   └── management/                     # Management commands  
├── logs/                               # Application log files (created at runtime)  
├── nginx/                              # Nginx configuration for production  
│   ├── Dockerfile  
│   └── nginx.conf  
├── docker-compose.yml                   # Development environment  
├── docker-compose.prod.yml              # Production environment  
├── Dockerfile                           # Production Docker image  
├── Dockerfile.dev                       # Development Docker image  
├── docker-entrypoint.sh                 # Container entrypoint script  
├── uwsgi.ini                           # uWSGI configuration  
├── requirements.txt                     # Production dependencies  
├── requirements-dev.txt                 # Development dependencies  
└── example.env                         # Environment variables template
```

Development Environment Setup

Option 1: Docker Compose (Recommended)

1. Clone the repository:

```
git clone <repository-url>  
cd tlux-backend
```

2. Create environment file:

```
cp example.env .env
```

3. Configure environment variables: Edit `.env` file with your development settings:

```
SECRET_KEY=your-secret-key-here  
DEBUG=True  
ALLOWED_HOSTS=localhost,127.0.0.1  
POSTGRES_DB=tlux  
POSTGRES_USER=tlux  
POSTGRES_PASSWORD=tlux
```

```
DATABASE_URL=postgresql://tlux:tlux@db:5432/tlux  
CORS_ORIGIN_ALLOW_ALL=True
```

4. Start the development environment:

```
docker-compose up --build
```

5. Access the application:

- API: <http://localhost:8000>
- Admin Panel: <http://localhost:8000/secret-admin/>
- API Documentation (Swagger): <http://localhost:8000/api/schema/swagger-ui/>
- API Documentation (ReDoc): <http://localhost:8000/api/schema/redoc/>

6. Run database migrations (if needed):

```
docker-compose exec web python manage.py migrate
```

7. Create a superuser (if needed):

```
docker-compose exec web python manage.py createsuperuser
```

8. Stop the environment:

```
docker-compose down
```

Option 2: Local Development (Without Docker)

1. Create a virtual environment:

```
python -m venv venv  
source venv/bin/activate # On Windows: venv\Scripts\activate
```

2. Install dependencies:

```
pip install -r requirements.txt  
pip install -r requirements-dev.txt
```

3. Set up PostgreSQL database:

- Create a PostgreSQL database
- Update `.env` with your database credentials

4. Configure environment variables:

```
cp example.env .env
# Edit .env with your local settings
```

5. Run migrations:

```
python manage.py migrate
```

6. Create a superuser:

```
python manage.py createsuperuser
```

7. Start the development server:

```
python manage.py runserver
```

Production Deployment

Prerequisites for Production

- Docker and Docker Compose installed on the server
- Domain name configured (optional, but recommended)
- SSL certificates (for HTTPS - recommended)

Deployment Steps

1. Clone the repository on the production server:

```
git clone <repository-url>
cd tlux-backend
```

2. Create production environment file:

```
cp example.env .env
```

3. Configure production environment variables: Edit `.env` with production settings:

```
SECRET_KEY=<generate-a-strong-secret-key>
DEBUG=False
ALLOWED_HOSTS=yourdomain.com,www.yourdomain.com,<server-ip>
POSTGRES_DB=tlux_prod
POSTGRES_USER=tlux_user
POSTGRES_PASSWORD=<strong-password>
DATABASE_URL=postgresql://tlux_user:<strong-
password>@db:5432/tlux_prod
CORS_ORIGIN_ALLOW_ALL=False
CORS_ALLOWED_ORIGINS=https://yourdomain.com,https://www.yourdomain.com
SERVER_NAME=yourdomain.com
```

Important: Generate a secure SECRET_KEY:

```
python -c "from django.core.management.utils import
get_random_secret_key; print(get_random_secret_key())"
```

4. Build and start production services:

```
docker-compose -f docker-compose.prod.yml up -d --build
```

5. Run database migrations:

```
docker-compose -f docker-compose.prod.yml exec web python manage.py
migrate
```

6. Create superuser:

```
docker-compose -f docker-compose.prod.yml exec web python manage.py
createsuperuser
```

7. Verify services are running:

```
docker-compose -f docker-compose.prod.yml ps
```

8. Check logs (if needed):

```
docker-compose -f docker-compose.prod.yml logs -f
```

Production Configuration Notes

- **Static Files:** Automatically collected by the entrypoint script and served by Nginx
- **Media Files:** Served by Nginx from the `/app/media` volume
- **Log Files:** Stored in `/app/logs` volume, accessible on host at `./logs/` (dev) or Docker volume (prod)
- **Database:** PostgreSQL data persists in a Docker volume (`postgres_data`)
- **uWSGI:** Configured with 4 processes and 2 threads per process
- **Nginx:** Handles static/media files and proxies API requests to uWSGI

SSL/HTTPS Setup (Recommended)

For production, configure SSL certificates:

1. Using Certbot (Let's Encrypt):

```
# Install Certbot
sudo apt-get update
sudo apt-get install certbot python3-certbot-nginx

# Obtain certificate (adjust nginx config first)
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

2. Update Nginx configuration to support HTTPS and redirect HTTP to HTTPS.

3. Restart Nginx service:

```
docker-compose -f docker-compose.prod.yml restart nginx
```

Maintenance Commands

• View logs:

```
# Docker container logs
docker-compose -f docker-compose.prod.yml logs -f [service_name]

# Application log files (on host)
tail -f logs/application.log
tail -f logs/error.log

# Application log files (in container)
docker-compose -f docker-compose.prod.yml exec web tail -f
/app/logs/application.log
```

• Restart services:

```
docker-compose -f docker-compose.prod.yml restart
```

- **Stop services:**

```
docker-compose -f docker-compose.prod.yml down
```

- **Update application:**

```
git pull
docker-compose -f docker-compose.prod.yml up -d --build
docker-compose -f docker-compose.prod.yml exec web python manage.py migrate
```

- **Backup database:**

```
docker-compose -f docker-compose.prod.yml exec db pg_dump -U tlux_user tlux_prod > backup.sql
```

- **Restore database:**

```
docker-compose -f docker-compose.prod.yml exec -T db psql -U tlux_user tlux_prod < backup.sql
```

- **View log file sizes:**

```
ls -lh logs/
```

- **Search logs:**

```
grep "ERROR" logs/application.log
grep "WARNING" logs/error.log
```

Environment Variables

Key environment variables (see `example.env` for full list):

Variable	Description	Default
----------	-------------	---------

Variable	Description	Default
SECRET_KEY	Django secret key	Required
DEBUG	Enable debug mode	True
ALLOWED_HOSTS	Comma-separated list of allowed hosts	[]
DATABASE_URL	PostgreSQL connection string	sqlite:///db.sqlite3
POSTGRES_DB	PostgreSQL database name	tlux
POSTGRES_USER	PostgreSQL username	tlux
POSTGRES_PASSWORD	PostgreSQL password	tlux
CORS_ORIGIN_ALLOW_ALL	Allow all CORS origins	True
CORS_ALLOWED_ORIGINS	Comma-separated allowed origins	http://localhost,http://localhost:3000
SERVER_NAME	Nginx server name	_
LOG_LEVEL	Log level (DEBUG, INFO, WARNING, ERROR, CRITICAL)	INFO
LOG_ALL_LEVELS	Capture all log levels	False
LOG_TO_CONSOLE	Enable console logging	True
LOG_TO_FILE	Enable file logging	False
LOG_FORMAT	Log format (verbose or json)	verbose
LOG_DIR	Directory for log files	logs
LOG_FILE_MAX_BYTES	Max log file size before rotation	10485760 (10MB)
LOG_FILE_BACKUP_COUNT	Number of backup log files	5

Variable	Description	Default
LOG_DB_QUERIES	Enable database query logging	False
CELERY_BROKER_URL	RabbitMQ connection URL	amqp://guest:guest@rabbitmq:5672//
CELERY_RESULT_BACKEND	Celery result backend URL	rpc://
RABBITMQ_USER	RabbitMQ username	guest
RABBITMQ_PASSWORD	RabbitMQ password	guest

API Documentation

Once the server is running, access the API documentation:

- **Swagger UI:** `/api/schema/swagger-ui/`
- **ReDoc:** `/api/schema/redoc/`
- **OpenAPI Schema (JSON):** `/api/schema/`

Database Migrations

Run migrations in development:

```
docker-compose exec web python manage.py migrate
```

Run migrations in production:

```
docker-compose -f docker-compose.prod.yml exec web python manage.py migrate
```

Admin Panel

- **Development:** Access Django admin at `/secret-admin/` (honeypot at `/admin/`)
- **Production:** Use the same paths as development

Security Note: The `/admin/` path is protected by `admin-honeypot` to prevent unauthorized access. Use `/secret-admin/` for actual administration.

Logging

The application includes a comprehensive logging system that supports both console and file logging with configurable log levels and formats.

Log Configuration

Logging is configured via environment variables in your `.env` file:

```
# Log level: DEBUG, INFO, WARNING, ERROR, CRITICAL
LOG_LEVEL=INFO

# Capture all log levels (overrides LOG_LEVEL when True)
LOG_ALL_LEVELS=False

# Enable console logging (recommended for development)
LOG_TO_CONSOLE=True

# Enable file logging (recommended for production)
LOG_TO_FILE=True

# Log format: 'verbose' for human-readable, 'json' for structured JSON
LOG_FORMAT=verbose

# Directory for log files
LOG_DIR=logs

# Log rotation settings
LOG_FILE_MAX_BYTES=10485760 # 10MB
LOG_FILE_BACKUP_COUNT=5

# Enable database query logging (for debugging)
LOG_DB_QUERIES=False
```

Log Files

When `LOG_TO_FILE=True`, the following log files are created in the `logs/` directory:

- **application.log**: General application logs
- **error.log**: Errors and critical messages (all levels if `LOG_ALL_LEVELS=True`)
- **django.log**: Django framework-specific logs
- **database.log**: Database query logs (when `LOG_DB_QUERIES=True`)

Accessing Logs

Development (Docker)

Logs are accessible both inside the container and on the host:

```
# View logs on host
tail -f logs/application.log
tail -f logs/error.log

# View logs inside container
docker-compose exec web tail -f /app/logs/application.log
```

```
# View all log files
ls -lh logs/
```

Production

Logs are stored in a Docker volume and can be accessed via:

```
# View logs from container
docker-compose -f docker-compose.prod.yml exec web tail -f
/app/logs/application.log

# Or access the volume directly
docker volume inspect drf-ready-to-go_logs_volume
```

Log Format

Verbose Format (default):

```
2025-12-23 07:41:43 [INFO] core.views:24 test_logging() - INFO: This is an
info message
```

JSON Format (requires `pythonjsonlogger` package):

```
{
  "asctime": "2025-12-23 07:41:43",
  "name": "core.views",
  "levelname": "INFO",
  "message": "This is an info message",
  "pathname": "/app/core/views.py",
  "lineno": 24,
  "funcName": "test_logging"
}
```

Using Logging in Your Code

```
from core.logging import get_logger

# Get a logger instance
logger = get_logger(__name__)

# Log at different levels
logger.debug("Debug message")
logger.info("Info message")
```

```
logger.warning("Warning message")
logger.error("Error message")
logger.critical("Critical message")

# Log with extra context
logger.info(
    "User action",
    extra={"user_id": 123, "action": "login"}
)
```

Log Rotation

Log files automatically rotate when they reach the configured maximum size (`LOG_FILE_MAX_BYTES`). The system keeps the specified number of backup files (`LOG_FILE_BACKUP_COUNT`).

Production Recommendations

For production environments:

1. Set `LOG_TO_FILE=True` to persist logs
2. Set `LOG_ALL_LEVELS=True` for comprehensive logging (if needed)
3. Use `LOG_FORMAT=json` for log aggregation tools (requires `pythonjsonlogger`)
4. Monitor log file sizes and disk space
5. Set up log rotation and retention policies
6. Consider integrating with log aggregation services (ELK, Splunk, etc.)

Celery Task Queue

This project uses [Celery](#) with [RabbitMQ](#) as the message broker for handling asynchronous and periodic tasks.

Features

- **Asynchronous Tasks:** Execute long-running operations in the background
- **Periodic Tasks:** Schedule recurring tasks using `django-celery-beat`
- **Task Monitoring:** Track task execution and results
- **Scalable:** Run multiple workers for increased throughput

Prerequisites

- RabbitMQ service (included in Docker Compose)
- Celery worker process
- Celery beat scheduler (for periodic tasks)

Running Celery Workers

Development (Docker)

Start the Celery worker:

```
# Start worker in foreground
docker-compose exec web celery -A config worker -l info

# Start worker in background
docker-compose exec -d web celery -A config worker -l info
```

Start the Celery beat scheduler (for periodic tasks):

```
# Start beat scheduler in foreground
docker-compose exec web celery -A config beat -l info

# Start beat scheduler in background
docker-compose exec -d web celery -A config beat -l info
```

Local Development (Without Docker)

1. Start RabbitMQ (if not using Docker):

```
# Using Docker for RabbitMQ only
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 \
  -e RABBITMQ_DEFAULT_USER=guest \
  -e RABBITMQ_DEFAULT_PASS=guest \
  rabbitmq:3-management-alpine
```

2. Start Celery worker:

```
celery -A config worker -l info
```

3. Start Celery beat scheduler:

```
celery -A config beat -l info
```

RabbitMQ Management UI

Access the RabbitMQ management interface:

- **URL:** <http://localhost:15672>
- **Username:** `guest` (default)
- **Password:** `guest` (default)

The management UI provides:

- Queue monitoring

- Connection status
- Message statistics
- Exchange and binding management

Sample Tasks

The project includes sample tasks in the `tasks` app:

Async Tasks

1. **Send Email Notification** (`tasks.send_email_notification`):

- Sends an email notification asynchronously
- Trigger via API: `POST /api/tasks/trigger-email/`

2. **Process Data Batch** (`tasks.process_data_batch`):

- Processes a batch of data items
- Trigger via API: `POST /api/tasks/trigger-batch/`

Periodic Tasks

1. **Health Check** (`tasks.periodic_health_check`):

- Runs periodically to check system health
- Configure via Django admin or `django-celery-beat`

2. **Cleanup Old Records** (`tasks.cleanup_old_records`):

- Cleans up old database records
- Configure via Django admin or `django-celery-beat`

Using Tasks in Your Code

Triggering Async Tasks

```
from tasks.tasks import send_email_notification, process_data_batch

# Trigger a task asynchronously
task = send_email_notification.delay(
    user_email="user@example.com",
    subject="Welcome",
    message="Welcome to our service!"
)

# Get task ID for tracking
print(f"Task ID: {task.id}")

# Trigger batch processing
task = process_data_batch.delay(
    batch_id="batch-123",
```

```
        items=["item1", "item2", "item3"]
    )
```

Creating Custom Tasks

```
from celery import shared_task
from core.logging import get_logger

logger = get_logger(__name__)

@shared_task(name="myapp.process_order")
def process_order(order_id: int) -> dict:
    """Process an order asynchronously."""
    logger.info(f"Processing order {order_id}")
    # Your processing logic here
    return {"status": "success", "order_id": order_id}
```

Configuring Periodic Tasks

1. Via Django Admin:

- Navigate to `/secret-admin/django_celery_beat/periodictask/`
- Create a new periodic task
- Select the task name (e.g., `tasks.periodic_health_check`)
- Configure the schedule (crontab or interval)

2. Via Code (in migrations or management commands):

```
from django_celery_beat.models import PeriodicTask, CrontabSchedule

schedule, _ = CrontabSchedule.objects.get_or_create(
    minute='0',
    hour='*/6', # Every 6 hours
)

PeriodicTask.objects.get_or_create(
    name='Health Check',
    defaults={
        'task': 'tasks.periodic_health_check',
        'crontab': schedule,
        'enabled': True,
    }
)
```

Environment Variables

Key Celery-related environment variables:

Variable	Description	Default
CELERY_BROKER_URL	RabbitMQ connection URL	amqp://guest:guest@rabbitmq:5672//
CELERY_RESULT_BACKEND	Result backend URL	rpc://
RABBITMQ_USER	RabbitMQ username	guest
RABBITMQ_PASSWORD	RabbitMQ password	guest

Monitoring Tasks

Check Task Status

```
from celery.result import AsyncResult
from config.celery import app

# Get task result
task_id = "your-task-id"
result = AsyncResult(task_id, app=app)

# Check status
print(result.state) # PENDING, SUCCESS, FAILURE, etc.
print(result.result) # Task return value
```

View Task Logs

```
# View worker logs
docker-compose logs -f web | grep celery

# View beat scheduler logs
docker-compose logs -f web | grep beat
```

Production Deployment

For production, run Celery workers and beat scheduler as separate services:

```
# Add to docker-compose.prod.yml
celery_worker:
  build:
    context: .
    dockerfile: Dockerfile
  command: celery -A config worker -l info
  env_file:
    - .env
```



```
depends_on:
  - db
  - rabbitmq

celery_beat:
  build:
    context: .
    dockerfile: Dockerfile
  command: celery -A config beat -l info
  env_file:
    - .env
  depends_on:
    - db
    - rabbitmq
```

Troubleshooting

Worker Not Processing Tasks

- Verify RabbitMQ is running: `docker-compose ps rabbitmq`
- Check broker URL in settings matches RabbitMQ configuration
- Review worker logs for errors

Periodic Tasks Not Running

- Ensure Celery beat scheduler is running
- Verify periodic tasks are enabled in Django admin
- Check beat scheduler logs for errors
- Run migrations: `python manage.py migrate` (creates beat tables)

Connection Errors

- Verify RabbitMQ credentials match environment variables
- Check network connectivity between services
- Ensure RabbitMQ health check passes

Testing

This project uses [pytest](#) and [pytest-django](#) for testing. The test suite includes coverage reporting and custom fixtures for API testing.

Prerequisites

Install development dependencies:

```
# Using pip
pip install -r requirements-dev.txt
```

```
# Using pipenv
pipenv install --dev
```

Running Tests

Run All Tests

```
# Basic test run
pytest

# With verbose output
pytest -v

# With coverage report
pytest --cov

# Show coverage report in terminal
pytest --cov --cov-report=term-missing
```

Run Specific Tests

```
# Run a specific test file
pytest tests/example_test.py

# Run tests matching a pattern
pytest -k "test_example"

# Run tests in a specific directory
pytest tests/
```

Run Tests with Markers

The project includes custom markers for organizing tests:

```
# Run only unit tests
pytest -m "unit"

# Run only API tests
pytest -m "api"

# Run only integration tests
pytest -m "integration"

# Exclude slow tests
pytest -m "not slow"
```

```
# Run multiple markers
pytest -m "unit and not slow"
```

Coverage Reports

```
# Generate HTML coverage report
pytest --cov --cov-report=html

# View the report (opens in browser)
# Open htmlcov/index.html

# Generate XML coverage report (for CI/CD)
pytest --cov --cov-report=xml
```

Test Configuration

Test configuration is defined in `pyproject.toml`:

- **Test Discovery:** Automatically discovers tests in `tests/` directory
- **Coverage:** Configured to track coverage for all project files
- **Database:** Uses `--reuse-db` flag for faster test runs
- **Markers:** Custom markers for test categorization

Available Fixtures

The project includes several reusable fixtures in `conftest.py`:

`api_client`

Unauthenticated API client for testing public endpoints:

```
def test_public_endpoint(api_client):
    response = api_client.get('/api/public/')
    assert response.status_code == 200
```

`authenticated_api_client`

Authenticated API client with a test user:

```
def test_protected_endpoint(authenticated_api_client):
    response = authenticated_api_client.get('/api/protected/')
    assert response.status_code == 200
```

`user`

Creates a test user instance:

```
def test_user_creation(user):  
    assert user.username == "testuser"  
    assert user.email == "testuser@example.com"
```

superuser

Creates a test superuser instance:

```
def test_admin_access(superuser):  
    assert superuser.is_superuser  
    assert superuser.is_staff
```

Writing Tests

Basic Test Structure

```
import pytest  
from django.contrib.auth import get_user_model  
  
User = get_user_model()  
  
@pytest.mark.django_db  
def test_example():  
    """Example test demonstrating basic pytest usage."""  
    assert User.objects.count() == 0  
    User.objects.create_user(  
        username="test",  
        email="test@example.com",  
        password="password123",  
    )  
    assert User.objects.count() == 1
```

API Testing Example

```
import pytest  
  
@pytest.mark.django_db  
@pytest.mark.api  
def test_api_endpoint(authenticated_api_client):  
    """Example API test using authenticated client."""  
    response = authenticated_api_client.get('/api/endpoint/')
```

```
assert response.status_code == 200
assert 'data' in response.json()
```

Using Markers

```
import pytest

@pytest.mark.django_db
@pytest.mark.unit
def test_unit_functionality():
    """Unit test example."""
    pass

@pytest.mark.django_db
@pytest.mark.integration
@pytest.mark.slow
def test_integration_functionality():
    """Integration test example (marked as slow)."""
    pass
```

Test Organization

Tests are organized in the `tests/` directory:

```
tests/
├── __init__.py
├── conftest.py           # App-specific fixtures
├── example_test.py       # Example tests (can be removed)
├── [app_name]/           # App-specific test modules
│   ├── test_models.py
│   ├── test_views.py
│   └── test_serializers.py
```

Running Tests in Docker

```
# Run tests in development container
docker-compose exec web pytest

# Run tests with coverage
docker-compose exec web pytest --cov

# Run specific test file
docker-compose exec web pytest tests/example_test.py
```

Continuous Integration

For CI/CD pipelines, use:

```
# Run tests with XML coverage report
pytest --cov --cov-report=xml --cov-report=term

# Run tests without coverage (faster)
pytest --no-cov
```

Best Practices

1. **Use descriptive test names:** Test function names should clearly describe what they test
2. **One assertion per test:** Keep tests focused on a single behavior
3. **Use fixtures:** Leverage existing fixtures or create app-specific ones
4. **Mark slow tests:** Use `@pytest.mark.slow` for tests that take longer
5. **Test isolation:** Each test should be independent and not rely on other tests
6. **Database transactions:** Use `@pytest.mark.django_db` for tests that need database access
7. **Mock external services:** Use `pytest-mock` for mocking external API calls

Development Workflow

1. **Make code changes** in your local environment
2. **Write tests** for new functionality
3. **Run tests** to ensure everything works: `pytest`
4. **Test locally** using Docker Compose
5. **Commit and push** changes to version control
6. **Deploy to production** following the deployment steps

Troubleshooting

Database Connection Issues

- Verify PostgreSQL container is running: `docker-compose ps`
- Check database credentials in `.env`
- Ensure `DATABASE_URL` matches your configuration

Static Files Not Serving

- Run `python manage.py collectstatic` in the container
- Verify Nginx configuration and volume mounts
- Check file permissions on static/media directories

Port Already in Use

- Change port mappings in `docker-compose.yml` if ports 8000 or 80 are in use
- Update Nginx configuration if port 80 is changed

Migration Errors

- Ensure database is accessible
- Check for conflicting migrations: `python manage.py showmigrations`
- Consider resetting database in development (backup first!)

Security Best Practices

-  Never commit `.env` file to version control
-  Use strong `SECRET_KEY` in production
-  Set `DEBUG=False` in production
-  Configure `ALLOWED_HOSTS` properly
-  Use HTTPS in production
-  Restrict CORS origins in production
-  Use strong database passwords
-  Regularly update dependencies
-  Monitor application logs for suspicious activity

Contributing

1. Create a feature branch
2. Make your changes
3. Test thoroughly
4. Submit a pull request

License

[Specify your license here]

Support

For issues and questions, please [create an issue](#) or contact the development team.